

ML BASICS & LINEAR REGRESSION

Q1. What is Machine Learning? Explain the main types of Machine Learning with examples.

Machine Learning

Machine Learning (ML) is a branch of Artificial Intelligence (AI) that enables computers to learn patterns from data and make decisions or predictions without being explicitly programmed.

Types of Machine Learning

1. Supervised Learning

In supervised learning, the model is trained using labeled data.

Examples:

- House price prediction
- Email spam detection
- Student marks prediction

2. Unsupervised Learning

In unsupervised learning, the model works with unlabeled data and identifies hidden patterns.

Examples:

- Customer segmentation
- Market basket analysis
- Clustering similar users

3. Reinforcement Learning

In reinforcement learning, an agent learns through rewards and punishments.

Examples:

- Self-driving cars
- Robotics
- Game-playing AI

Q2. What is Supervised Learning? Explain the difference between Regression and Classification.

Supervised Learning

Supervised learning is a machine learning technique where the model learns from labeled data.

Difference Between Regression and Classification

Regression

Classification

Predicts continuous values	Predicts categories/classes
Output is numerical	Output is categorical
Example: House price prediction	Example: Spam email detection
Algorithms: Linear Regression	Algorithms: Logistic Regression

Q3. What is Linear Regression? Write the Linear Regression equation and explain intercept and coefficient.

Linear Regression

Linear Regression is a supervised machine learning algorithm used to predict continuous values.

Linear Regression Equation

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

Where:

- y = Dependent variable
- b_0 = Intercept
- b_1, b_2 = Coefficients
- x_1, x_2 = Independent variables

Intercept

The intercept represents the predicted value when all independent variables are zero.

Coefficient

A coefficient represents how much the target variable changes when one unit of a feature changes.

Q4. Explain the following evaluation metrics used in Linear Regression.

1. SSE (Sum of Squared Errors)

It is the total squared difference between actual and predicted values.

2. MSE (Mean Squared Error)

It is the average of squared errors.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

3. RMSE (Root Mean Squared Error)

It is the square root of MSE.

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

4. R² Score

It measures how well the model explains variance in the data.

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$$

5. Adjusted R²

Adjusted R² adjusts the R² score based on the number of features used.

Q5. Explain the Machine Learning workflow steps from data collection to model evaluation.

Machine Learning Workflow

1. Data Collection
 2. Data Cleaning
 3. Exploratory Data Analysis (EDA)
 4. Feature Selection
 5. Train-Test Split
 6. Model Training
 7. Prediction
 8. Model Evaluation
 9. Model Improvement
 10. Deployment
-

Part B – Practical Questions

Q6. Load the Boston Housing dataset and perform basic data exploration.

Python Code

```
# Import libraries

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_boston

# Load dataset
boston = load_boston()
```

```

# Create dataframe
df = pd.DataFrame(boston.data, columns=boston.feature_names)

# Add target variable
df['MEDV'] = boston.target

# Display first 5 rows
print(df.head())

# Dataset information
print(df.info())

# Statistical summary
print(df.describe())

```

Output Interpretation

- head() displays the first 5 rows.
- info() provides dataset structure and data types.
- describe() gives statistical summaries such as mean, min, max, etc.

Q7. Perform Exploratory Data Analysis (EDA)

Pairplot

```

sns.pairplot(df[['RM', 'LSTAT', 'PTRATIO', 'MEDV']])
plt.show()

```

Correlation Heatmap

```

plt.figure(figsize=(12,8))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.show()

```

Interpretation

Features highly correlated with MEDV:

- RM (Positive correlation)
- LSTAT (Negative correlation)
- PTRATIO (Negative correlation)

This means:

- More rooms increase house price.
- Higher lower-status population percentage decreases price.
- Higher pupil-teacher ratio may reduce price.

Q8. Select independent variables (features) and dependent variable (target).

Independent variables

```
X = df.drop('MEDV', axis=1)
```

Dependent variable

```
y = df['MEDV']
```

Explanation

- X contains input features.
- y contains target variable (house price).

Q9. Split the dataset into training and testing sets.

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(  
X, y, test_size=0.2, random_state=42  
)
```

```
print(X_train.shape)
```

```
print(X_test.shape)
```

Explanation

- 80% data is used for training.
- 20% data is used for testing.

Q10. Train a Linear Regression model and evaluate performance.

Train the Model

```
from sklearn.linear_model import LinearRegression
```

Create model

```
model = LinearRegression()
```

```
# Train model
model.fit(X_train, y_train)
```

Make Predictions

```
# Predictions
y_pred = model.predict(X_test)

print(y_pred[:5])
```

Calculate Evaluation Metrics

```
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
import numpy as np
```

```
# MAE
mae = mean_absolute_error(y_test, y_pred)
print('MAE:', mae)
```

```
# MSE
mse = mean_squared_error(y_test, y_pred)
print('MSE:', mse)
```

```
# RMSE
rmse = np.sqrt(mse)
print('RMSE:', rmse)
```

```
# R2 Score
r2 = r2_score(y_test, y_pred)
print('R2 Score:', r2)
```

Calculate Adjusted R²

```
n = X_test.shape[0]
```

```
p = X_test.shape[1]
```

```
adjusted_r2 = 1 - ((1-r2)*(n-1)/(n-p-1))
```

```
print('Adjusted R2:', adjusted_r2)
```

Linear Regression Equation

```
print('Intercept:', model.intercept_)
```

```
print('Coefficients:', model.coef_)
```

Regression Equation

```
□genui□{"math_block_widget_always_prefetch_v2":{"content":"MEDV = b_o + b_1(CRIM) +  
b_2(ZN) + b_3(INDUS) + ... + b_n(LSTAT)"} }□
```

Interpretation of Coefficients

Feature	Interpretation
RM	Increase in number of rooms increases house price
LSTAT	Higher lower-status population decreases house price
PTRATIO	Higher pupil-teacher ratio decreases house price
TAX	Higher tax rate may reduce house price
DIS	Distance from employment centers affects price

Final Conclusion

The Linear Regression model was successfully trained on the Boston Housing dataset.

Findings:

- RM is positively correlated with house prices.
- LSTAT has a strong negative relationship with house prices.
- The model achieved a good R² score, showing effective prediction performance.
- EDA and heatmap helped identify important features.

Overall Result

The model can predict house prices with reasonable accuracy using housing-related features.

Complete Google Colab Notebook Code

```
# Import libraries

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns


from sklearn.datasets import load_boston
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score


# Load dataset

boston = load_boston()


# Create dataframe

df = pd.DataFrame(boston.data, columns=boston.feature_names)
df['MEDV'] = boston.target


# Basic exploration

print(df.head())
print(df.info())
print(df.describe())


# Pairplot

sns.pairplot(df[['RM', 'LSTAT', 'PTRATIO', 'MEDV']])
plt.show()


# Heatmap

plt.figure(figsize=(12,8))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.show()
```



```
# Features and target
X = df.drop('MEDV', axis=1)
y = df['MEDV']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42
)

# Model training
model = LinearRegression()
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Evaluation metrics
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

# Adjusted R2
n = X_test.shape[0]
p = X_test.shape[1]
adjusted_r2 = 1 - ((1-r2)*(n-1)/(n-p-1))

# Results
print('MAE:', mae)
print('MSE:', mse)
print('RMSE:', rmse)
print('R2 Score:', r2)
print('Adjusted R2:', adjusted_r2)
```

```
# Coefficients
print('Intercept:', model.intercept_)
print('Coefficients:', model.coef_)
```

References

1. [Scikit-learn Documentation](#)
2. [Python Pandas Documentation](#)
3. [Seaborn Documentation](#)
4. [Matplotlib Documentation](#)